CEFET/RJ - Centro Federal de Educação Tecnológica Celso Suckow da Fonseca
Unidade Maracanã

Curso	Período	Departamento	Prova
Ciência da Computação	**2º**	**CCGBCC**	P1 (X) P2 ( ) PF ( ) P3 ( )

Disciplina	Professora	Data
Estruturas de Dados	**Myrna Amorim**	**22/04/26**

Nome	Nota

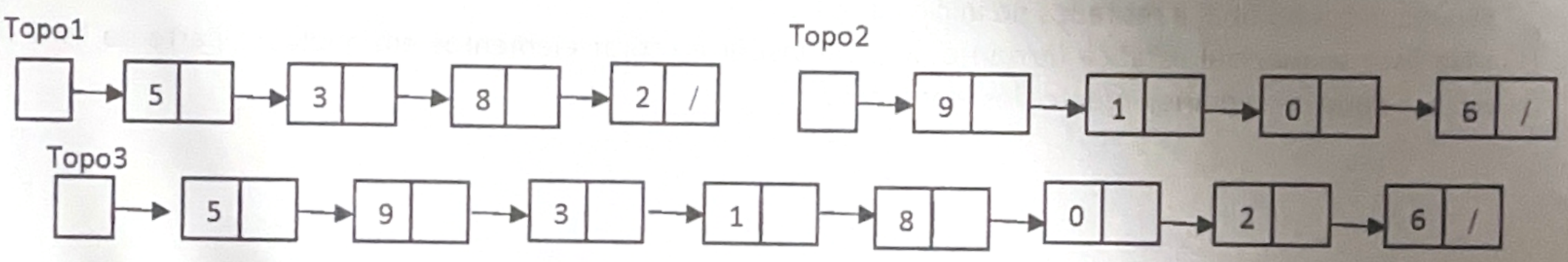
Importante:

1. ***Início: 12h50min Término: 14h40min***
2. *Leia atentamente toda a questão antes de iniciar.*
3. *A resposta deverá ser à **caneta** (azul ou preta).*
4. *PROIBIDO USAR CELULAR DURANTE A PROVA.*
5. *Coloque sempre o número da questão na folha de resposta.*
6. *As questões poderão ser respondidas fora de ordem, desde que seja obedecido o item 5.*

---

1. (2.0) Considere duas listas simplesmente encadeadas já preenchidas e de mesmo tamanho apontadas por dois ponteiros denominados Topo1 e Topo2, respectivamente. Elabore uma função que crie os nós de uma terceira lista simplesmente encadeada que contenha os valores intercalados das listas anteriores, iniciando pela lista apontada por Topo1. Informe e explique a complexidade de pior caso da função elaborada por você.

   Exemplo:



2. (2.0) Você foi contratado para implementar um app que cria uma lista de reprodução musical (*playlist*). Para isso, você deverá usar como estrutura de dados uma lista simplesmente encadeada. Cada música é representada por um nó da lista contendo os seguintes campos: código da música (int), tempo de duração em segundos (int) e o ponteiro para o próximo nó. Considere que a lista de músicas já esteja preenchida. Elabore as seguintes funções:

   a. (1.0) **reproduzirPlaylist()**
      - Mostrar (listar) todas os códigos das músicas e o tempo de duração de cada uma delas.

   b. (1.0) **buscarMusica()**
      - Percorrer a *playlist* procurando uma música pelo código.
        - Caso a música esteja na lista, então mostre o tempo de duração da música solicitada.
        - Caso a música não esteja na lista, mostre mensagem que não encontrou ou retorne um valor falso (ou 0).

```
struct musica
{
    int cod;
    int tempo;
    struct musica* prox;
};
```

3. (2.0) Usando como referência a explicação do algoritmo a seguir, faça uma função que ordene uma lista linear estática com 50 elementos inteiros. Informe a complexidade de pior caso do algoritmo desenvolvido.

O algoritmo percorre todo o array e a cada comparação entre os elementos, verifica e, caso necessário, marca o menor valor da lista. Ao chegar no último elemento e após realizar a verificação final, coloca o menor elemento selecionado na primeira posição da lista. Depois, percorre o array e seleciona o segundo menor valor da lista seguindo a mesma regra, para colocá-lo na segunda posição. Faz isso com o terceiro menor valor da lista e assim sucessivamente até que todos os elementos da lista sejam usados e que a lista fique classificada em ordem crescente.

4. (1.5) A busca ou pesquisa é uma das áreas mais importantes em Ciência da Computação. Em Estruturas de Dados vimos dois tipos de busca que possuem características distintas para encontrar um elemento numa lista linear. Baseado nas explicações de aula, faça:
   a. (1.0) implemente uma função que percorra uma lista linear, com complexidade de pior caso O(log n), e que retorne o valor -1 caso o elemento não esteja na lista ou o número da posição caso o elemento esteja na lista.
   b. (0.5) Que condição é necessária para usar busca binária que não se aplica à busca sequencial?

5. (2.5) Relacione V (verdadeiro) ou F (falso) para cada alternativa abaixo. **Caso a alternativa seja falsa, justifique o erro. Não serão consideradas as alternativas sem justificativa ou com justificativas incorretas.**

( ) Os algoritmos MergeSort e QuickSort utilizam uma técnica denominada divisão e conquista, onde o *array* é subdividido em partes menores até que não seja mais possível realizar a divisão. A principal diferença entre eles é que o QuickSort intercala as soluções obtidas para chegar à solução final, enquanto o MergeSort faz apenas concatenações.

( ) Apesar de ser mais eficiente, o algoritmo QuickSort possui a mesma complexidade de pior caso dos algoritmos BubbleSort e InsertionSort, ou seja, $O(n^2)$.

( ) Uma das vantagens de uma lista simplesmente encadeada em relação a um *array* é a possibilidade de buscar um elemento diretamente na lista (busca indexada).

( ) Fila e Pilha são políticas de inclusão e remoção numa lista que pode ser estática (array) ou dinâmica (encadeada). A diferença entre essas políticas é a forma como são realizadas a inclusão e a remoção de elementos. Numa Pilha, os elementos são incluídos no topo e retirados no final da lista. Numa Fila, os elementos são incluídos no final e retirados no início da lista.

( ) Na lista sequencial estática (*array*) é possível inserir e retirar elementos em qualquer parte da lista sem a necessidade de remanejamento dos elementos.